VULNERABILITY ASSESSMENT REPORT

Target: OWASP Juice Shop (practice application)

Prepared by: EDWIN BAIJU

Future Interns, Cyber Security Internship, Task 1

October 2026

Executive Summary

A security assessment was carried out on a web application to find weaknesses that attackers could use. The application tested was OWASP Juice Shop, a deliberately vulnerable practice app run on the tester's own machine, so no third-party system was touched.

The assessment found 10 issues: 0 High, 4 Medium, 3 Low and 3 Informational. No critical flaws were found by this passive review, but several missing security protections would make attacks easier. All of them can be fixed with simple server settings.

Overall risk rating: **MEDIUM**

Scope and Method

Target: http://localhost:3000 (local test environment)

Tools used:

- Nmap: checked which network ports are open and what software runs on them
- OWASP ZAP (passive scan): reviewed the site's responses for weaknesses while browsing through the app
- Browser DevTools and curl: inspected the security headers sent by the server

Steps:

1. Scanned the server with Nmap to find open ports and services
2. Checked the security headers using curl
3. Explored the application through ZAP and reviewed the alerts it raised
4. Classified each finding as Low, Medium or High and wrote fixes in plain language

Limitations: Only passive checks were used. No attacks were launched against the application, so deeper flaws may exist that this review did not find.

Risk Rating Guide and Summary

Risk rating guide:

- High: serious weakness that could lead to stolen data or system takeover. Fix immediately.

- Medium: weakness that makes attacks easier. Fix soon
- Low: minor weakness or information leak. Fix when possible.
- Informational: observation with no direct risk.

Summary of findings:

- High: 0
- Medium: 4
- Low: 3
- Informational: 3

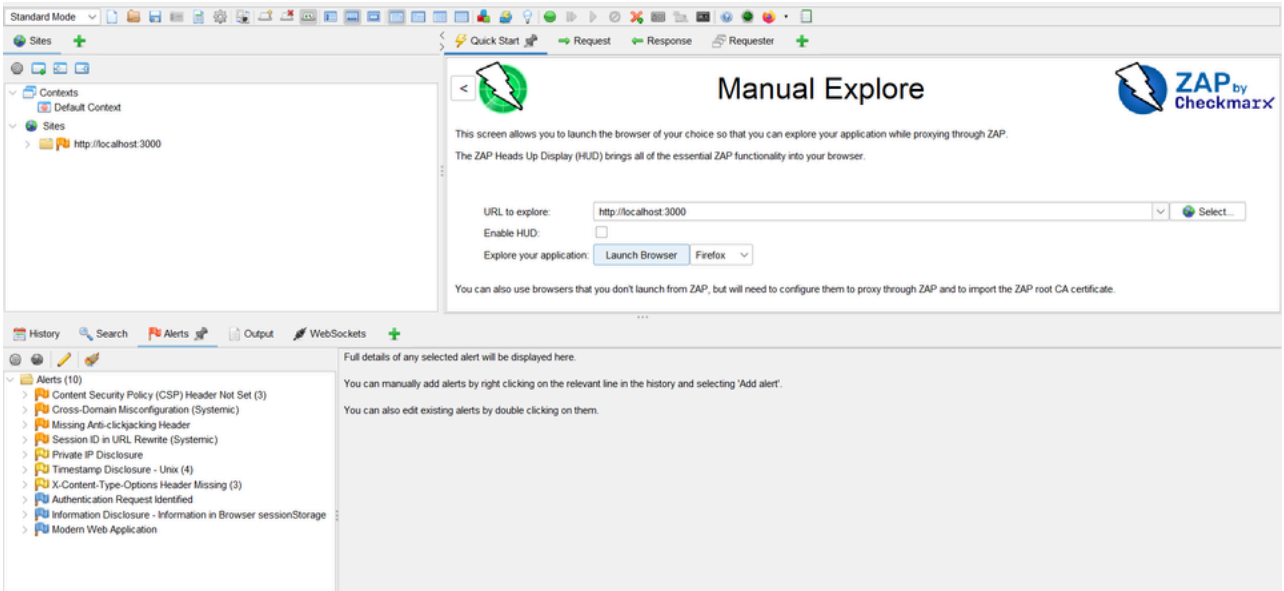


Figure 1: ZAP alerts list

Detailed Findings: Medium Risk

FINDING 1: Content Security Policy (CSP) Header Not Set

Risk: MEDIUM

What it is: The website does not tell browsers which scripts and resources it trusts.

Business impact: If an attacker injects a malicious script (cross-site scripting, XSS), the browser will run it. This could steal customer sessions or personal data.

Fix: Add a Content-Security-Policy header on the web server that allows scripts only from the company's own domain.

FINDING 2: Cross-Domain Misconfiguration

Risk: MEDIUM

What it is: The server sends "Access-Control-Allow-Origin: *", which lets any website read its responses

Business impact: A malicious website could request data from this application on behalf of a visitor and read the result.

Fix: Replace the wildcard (*) with a list of specific trusted domains.

Detailed Findings: Medium Risk (continued)

FINDING 3: Missing Anti-clickjacking Header

Risk: MEDIUM

What it is: Some pages do not stop other sites from loading them inside a hidden frame.

Business impact: Attackers can trick users into clicking buttons they cannot see (clickjacking), for example to confirm a purchase or change a setting.

Fix: Add the X-Frame-Options header (DENY or SAMEORIGIN) or the CSP frame-ancestors directive to all pages.

FINDING 4: Session ID in URL Rewrite

Risk: MEDIUM

What it is: Session identifiers appear inside web addresses.

Business impact: Web addresses are saved in browser history, logs and shared links. Anyone who sees the address could take over the user's session.

Fix: Keep session IDs in secure cookies only, never in the URL.

Evidence Screenshots

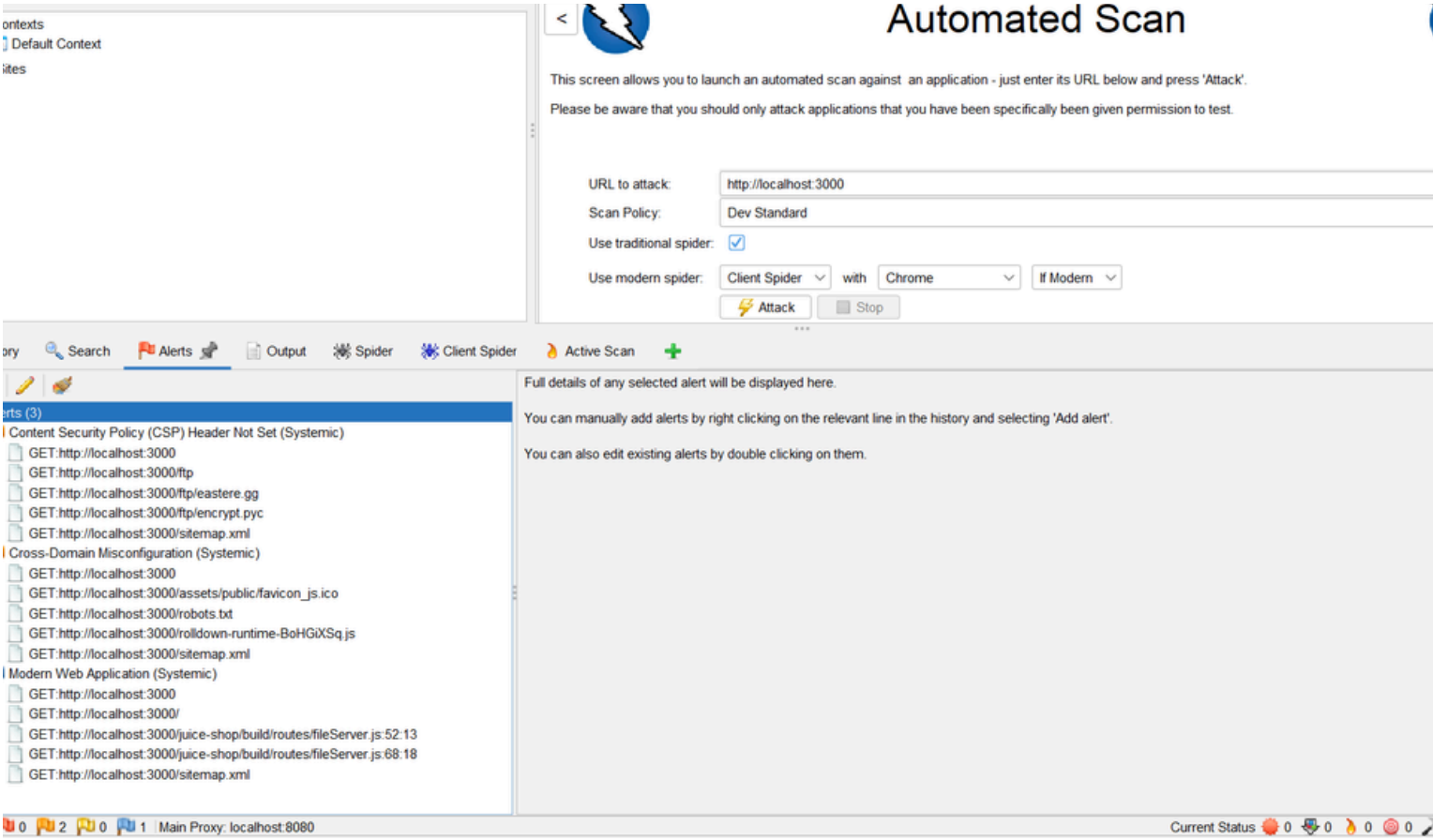


Figure 2: ZAP alert details for Finding 1 (CSP header not set)

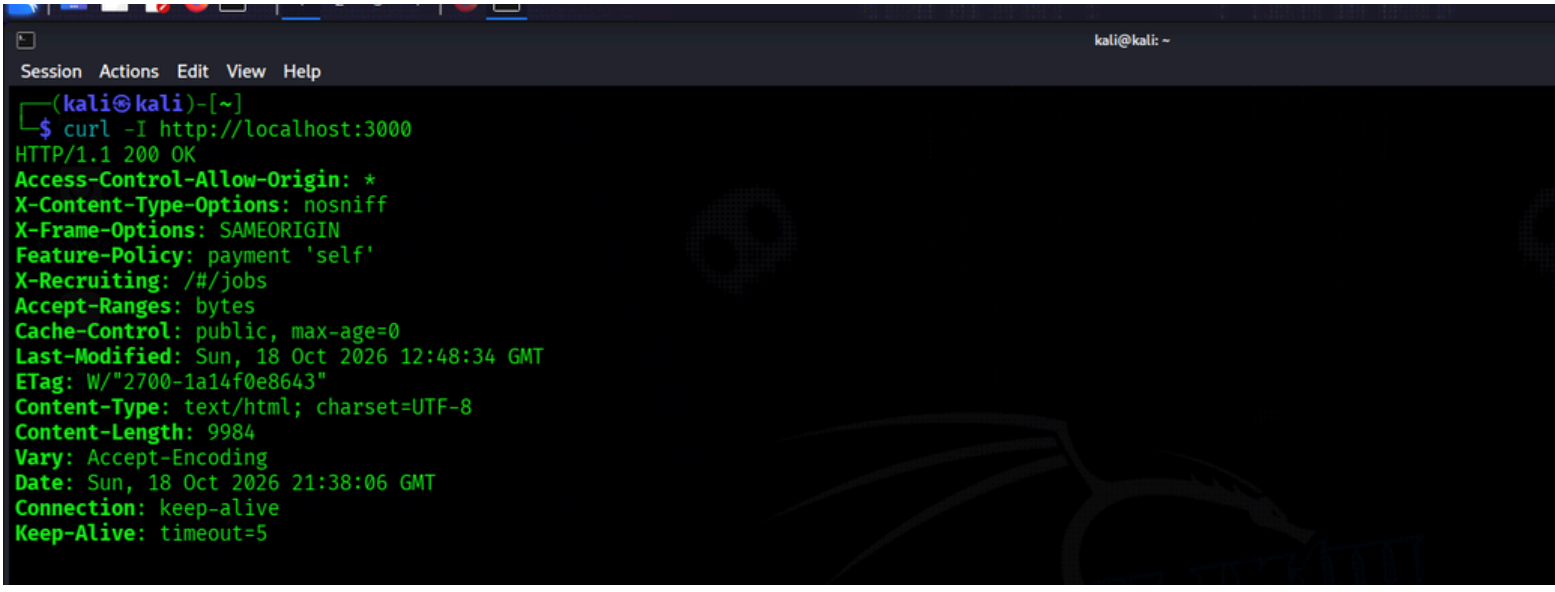


Figure 3: curl response headers supporting Finding 2 (Access-Control-Allow-Origin: *)

Detailed Findings: Low Risk

FINDING 5: Private IP Disclosure
Risk: LOW
What it is: The application reveals internal network (private) IP addresses in its responses.
Business impact: Attackers learn how the internal network is arranged, which helps them plan further attacks.
Fix: Remove internal addresses from responses, error messages and comments.
Evidence: see the ZAP alerts list (Figure 1).

FINDING 6: Timestamp Disclosure (Unix)
Risk: LOW
What it is: Internal time values appear in the application's responses.
Business impact: On its own it is harmless, but it gives attackers extra details about how the system works.
Fix: Remove timestamps that users do not need to see.
Evidence: see the ZAP alerts list (Figure 1).

FINDING 7: X-Content-Type-Options Header Missing
Risk: LOW
What it is: Some responses lack the "nosniff" instruction that stops browsers from guessing file types.
Business impact: A browser might treat a harmless-looking file as a script and run it.
Fix: Add the header X-Content-Type-Options: nosniff to all responses.
Evidence: see the ZAP alerts list (Figure 1).

Detailed Findings: Informational

FINDING 8: Authentication Request Identified

Risk: INFORMATIONAL

What it is: ZAP found the login request of the application.

Business impact: No direct risk. It marks where deeper login testing (password rules, lockout, brute-force protection) should be done.

Fix: Make sure the login has lockout or rate limiting and uses HTTPS.

Evidence: see the ZAP alerts list (Figure 1).

FINDING 9: Information Disclosure in Browser sessionStorage

Risk: INFORMATIONAL

What it is: The application stores data in the browser's session storage.

Business impact: If an attacker manages to run a script in the browser, they could read this data.

Fix: Avoid storing sensitive data in browser storage. Use secure, HttpOnly cookies instead.

Evidence: see the ZAP alerts list (Figure 1).

FINDING 10: Modern Web Application

Risk: INFORMATIONAL

What it is: The site relies heavily on JavaScript, so basic scanners can miss some pages.

Business impact: No direct risk, but a deeper test may find more issues.

Fix: Run a full authenticated scan in a future assessment.

Evidence: see the ZAP alerts list (Figure 1).

Additional Observations (Nmap and curl)

Findings from the Nmap scan and curl header check:

- Plain HTTP: The application runs on port 3000 over HTTP with no encryption, so data sent to it could be read on the network. Fix: serve the application over HTTPS. - No HSTS header: The Strict-Transport-Security header was not seen. Fix: add HSTS once HTTPS is enabled. - Good practice found: X-Frame-Options: SAMEORIGIN and X-Content-Type-Options: nosniff were present on the main page.

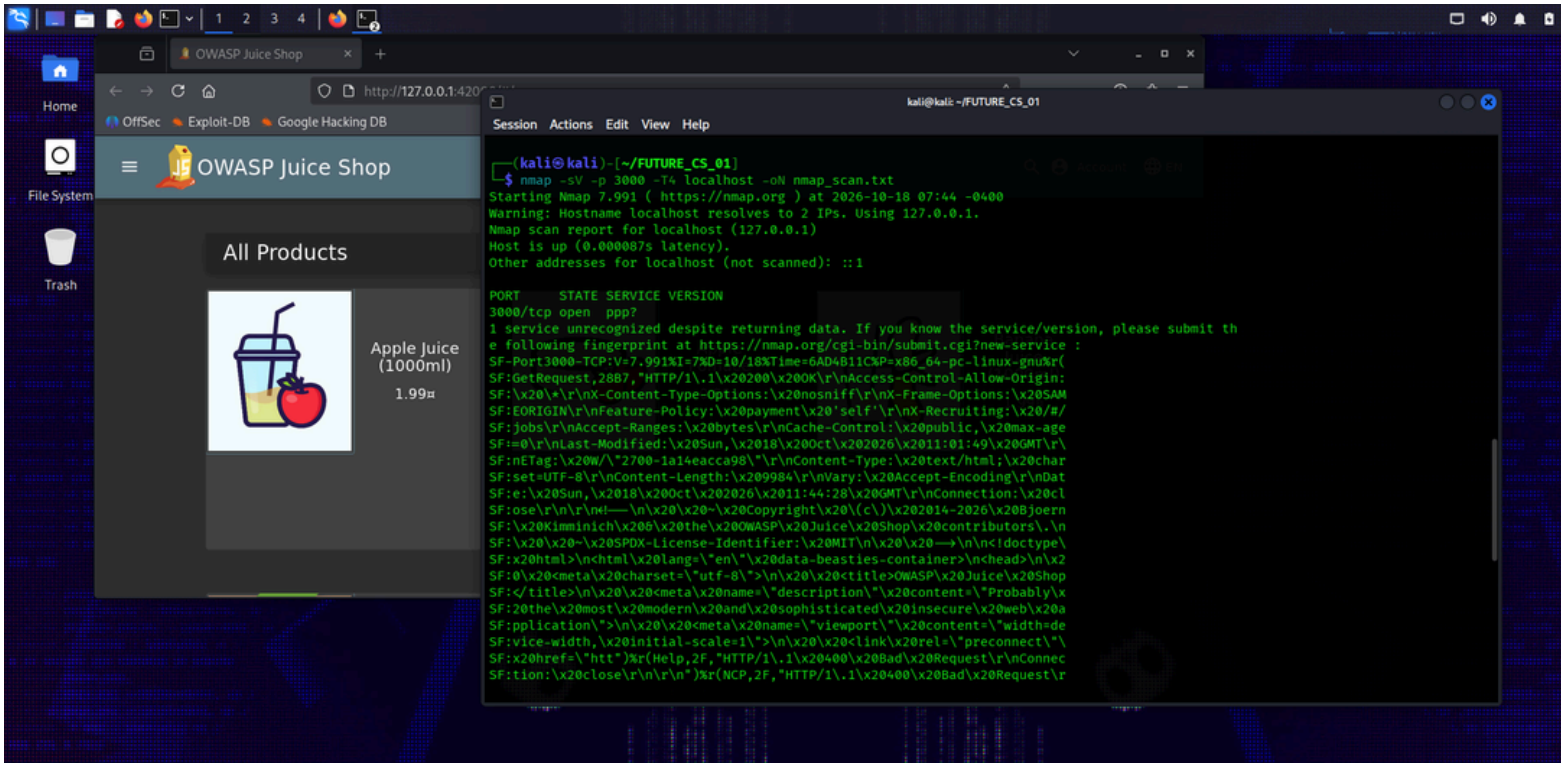


Figure 4: Nmap scan of localhost port 3000

Action Plan and Conclusion

Recommended action plan:

- Fix first (Medium): add a Content-Security-Policy header, restrict cross-domain access, add anti-clickjacking protection, and remove session IDs from URLs.
- Fix next (Low): remove internal IP and timestamp leaks, and add the nosniff header to all responses.
- Ongoing: enable HTTPS and HSTS, run regular scans, and repeat the assessment after the fixes.

Conclusion:

The assessment found no High-risk issues in this passive review, but ten weaknesses were identified, four of them Medium. They are mostly missing security headers and information leaks, which can be fixed quickly with server configuration. After fixing them, the application should be tested again, including deeper active testing, to confirm the improvements.

Note: The target was a deliberately vulnerable application running on the tester's own machine. No third-party systems were tested.